Auditing recall notification dates before building a remedy-time model

A twelve-campaign source audit and typed-event implementation exercise

James N. Jean Independent research, 30 October 2026

Source capture: 31 October 2026 UTC. Not peer reviewed. No agency endorsement.

The practical decision

A recall dashboard can display a notification date correctly and still supply the wrong label for a remedy-time model. Keep the event role, date precision, planned/reported state and evidence identifier together before allowing that date into an analysis.

This finite audit retains all 12 campaigns numbered 23V001000–23V012000. Six dates match current generic notification assertions; one conflicts with a current reported date; five lack a supported past-event binding under conservative coding. No physical first-availability date is established. The deliverable is a complete case table and a local guard that rejects unsupported event conversions.

Recommended action. Use the typed notice as a notice. Investigate the conflicting source date and keep remedy availability unknown until separate evidence establishes it. A smaller or cleaner-looking label table is not evidence of a valid remedy-time target.

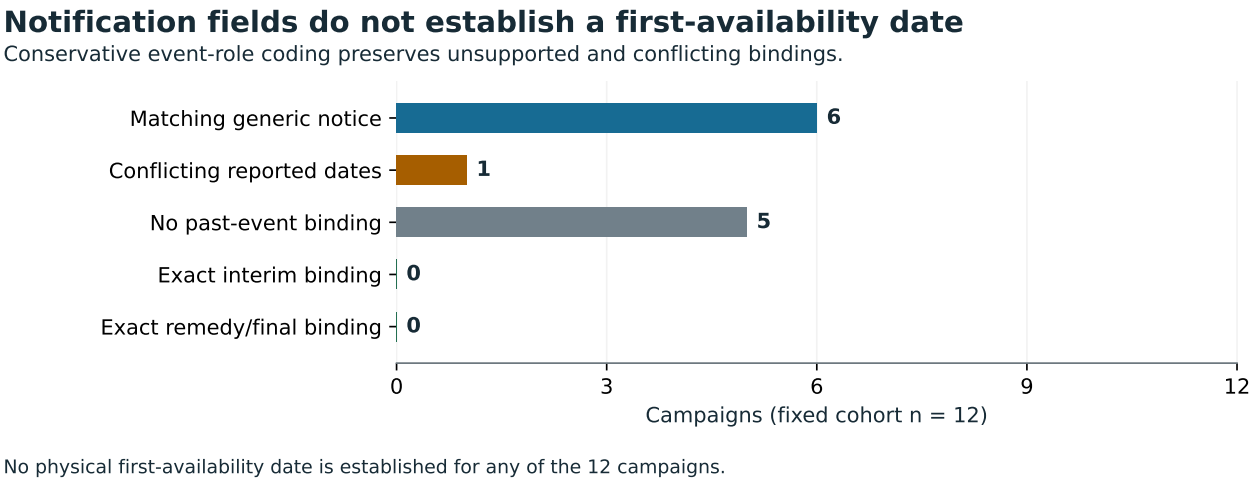


Figure 1. All 12 fixed campaigns. There are 6 matching generic notices, 1 conflict and 5 unresolved bindings. Exact dated interim and remedy/final bindings are both zero under the coding rules. These are conservative evidence categories, not national error-rate estimates.

Scope. This is descriptive source measurement and defensive software, with no fitted model, tuned threshold, prediction, repair-delivery measurement or safety validation. The original proposal to predict time to final remedy from initial Part 573 information remains unexecuted: both the outcome label and initial-vintage gates are unresolved.

Every campaign is retained

Campaign	Flat ODATE	Current API date	API state	Binding	Days
23V001	2023-01-31	2023-03-01	expected	unresolved	n/a
23V002	2023-01-17	2023-01-17	past	generic match	0
23V003	2023-02-16	2023-03-10	expected	unresolved	n/a
23V004	2023-01-20	2023-01-20	past	generic match	0
23V005	2023-02-07	2023-03-02	past	conflict	-23
23V006	2023-02-09	2023-02-01	expected	unresolved	n/a
23V007	2023-03-10	2023-03-17	expected	unresolved	n/a
23V008	2023-03-17	2023-03-17	past	generic match	0
23V009	2023-02-01	2023-02-01	past	generic match	0
23V010	2023-02-23	2023-02-23	past	generic match	0
23V011	2023-02-22	2023-02-22	past	generic match	0
23V012	2023-02-27	2023-03-15	expected	unresolved	n/a

Table 1. *Past* means the current API explicitly says letters were mailed or notification began. *Expected* remains a plan. Days is flat ODATE minus a reported API notice date, in calendar days; n/a means no comparable past assertion. A zero is source-date equality, not zero repair delay. These are current attributed assertions, not independently verified delivery.

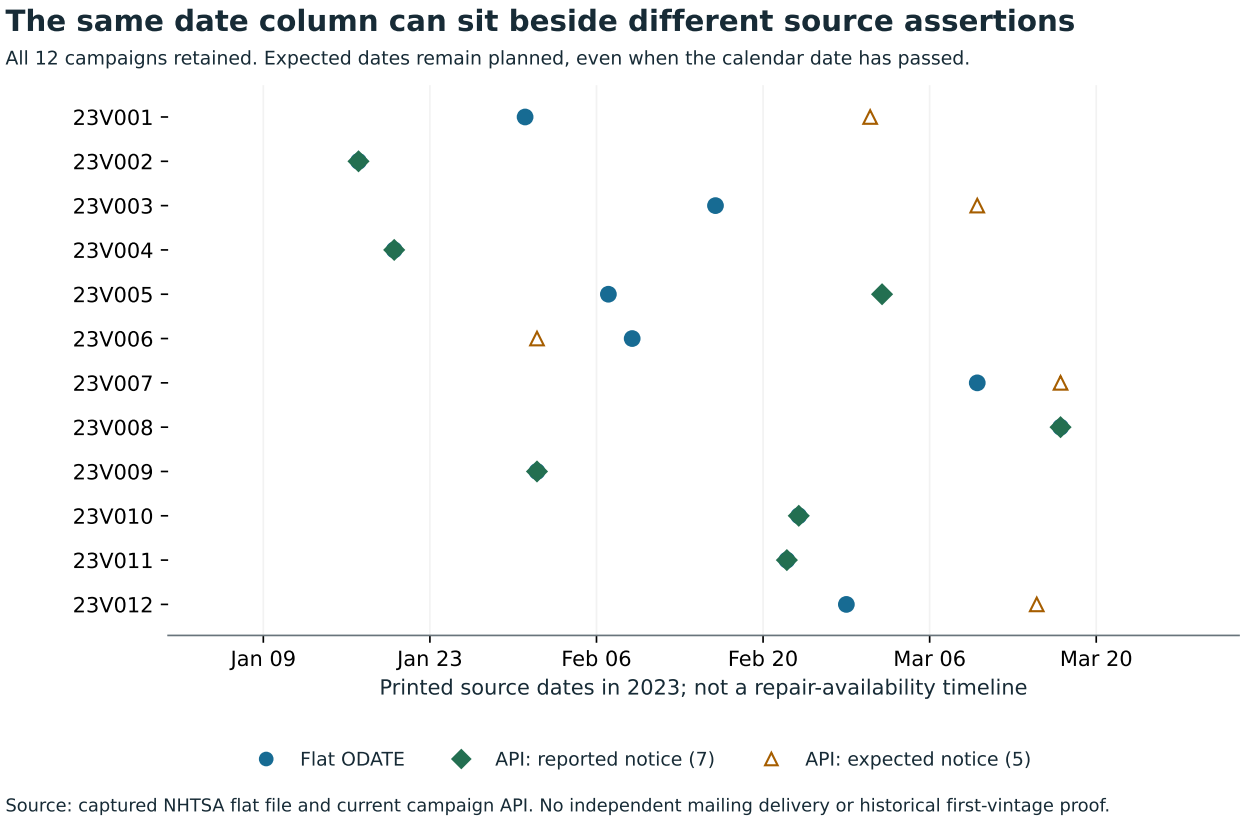


Figure 2. The same generic flat-file date is shown beside reported or expected API assertions. Coincident markers indicate equal dates. All cases remain visible; planned dates do not become actual events just because the calendar day has passed.

The unit is the campaign. The 182 matching flat-file model rows and 113 API records are different source representations; neither is an independent sample of campaigns.

What document review changes

The source conflict: 23V005. Flat ODATE is 2023-02-07, whereas the current API says owner notification letters were mailed 2023-03-02. A dated owner letter also says 2 March 2023. The signed discrepancy is -23 days. Different phases or later revisions could explain it; this audit does not determine that the agency field is erroneous. The guard holds the conflicting binding for review.

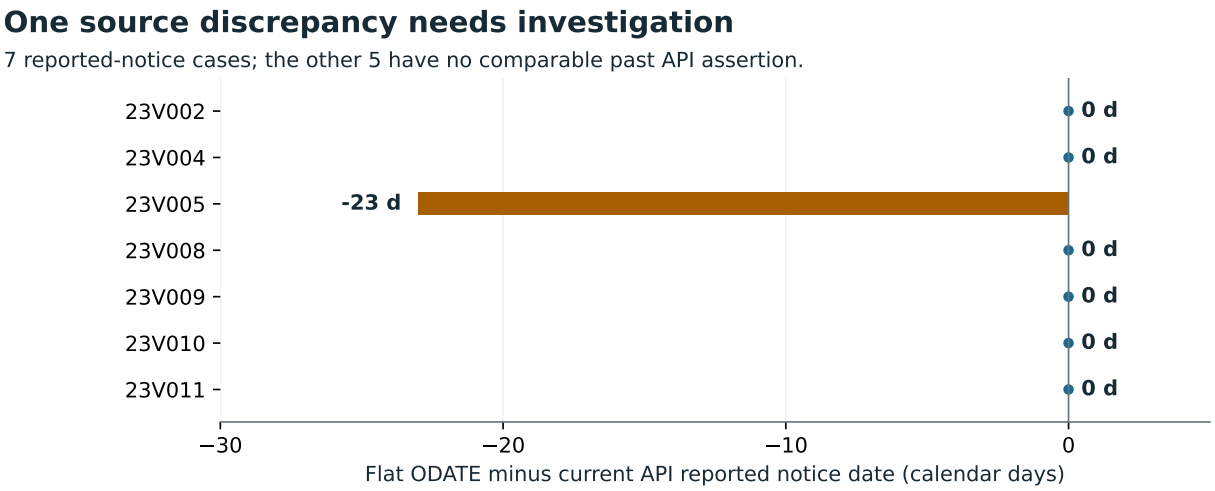


Figure 3. All 7 campaigns with a past API notification assertion. The other 5 have no comparable past assertion. This is a source-date comparison, not a distribution of actual remedy delay.

A matching letter date is insufficient: 23V003. Its owner letter is dated 16 February 2023, matching ODATE. The API still describes expected notification on 10 March. The printed letter day alone does not prove mailing, so the past-event binding remains unresolved. The attached service notice has a release date; that is another event role.

Interim is not final: 23V010. The issued interim template explicitly says the remedy is not currently available, but has no printed exact date. Later Part 573 amendments describe phased notification beginning 23 February for 2022–2023 vehicles, plus other planned model-year phases. The generic ODATE match cannot identify an exact interim or final notice, or first physical remedy availability.

An availability assertion can still lack a day: 23V011. The letter says *Remedy is Available*, but is marked SAMPLE and dated only February 2023. The API's reported 22 February mailing remains a generic notice assertion; the sample does not establish a first-availability day.

Channel timing matters: 23V012. The 27 February letter says a dealer upgrade can be performed, while OTA release will not be available until mid-March. Collapsing these statements into one remedy-availability date loses their meaning.

Sources, coding and implementation

The campaign numbers were fixed before capture. Selection was source informed: a prior API example and one Volvo notification date were already visible. There is no blinded or representative-cohort claim.

The official campaign lists supplied 197 PDF links. A metadata filter captured 64 files. Four envelope templates were retained privately but excluded before semantic coding, leaving 60 eligible documents and 170 pages: 29 templated Part 573 reports, twelve acknowledgements, eighteen owner/renotification letters and one supplemental report. Every eligible document has a public factual adjudication row with native hash, type, page, date precision, event state, short evidence and unknowns. Planned ranges, amendments, generic templates and channel-dependent statements stay distinct.

A default PDF reader reordered header columns. A source-flow extraction repair resolved all 58 structured cell comparisons between pypdf and pdfplumber. Relevant native page regions were visually inspected; not every page was visually inspected. Nine cells needed native-page visual checks because of raster content or extraction differences. One narrative date page was corrected before comparison. Earlier failed receipts are retained privately.

The code accepts an exact date and source evidence, checks campaign identity and certainty, and preserves the original event role. The six source-backed exercises produce:

Input request	Outcome
Reported generic notice with matching evidence	Accept; retain generic notice role
Same notice presented as physical availability	Reject unsupported role conversion
Expected notice presented as a reported event	Reject planned-as-reported input
Undated interim template	Reject missing exact date
Conflicting current notification sources	Reject ambiguous binding
Missing evidence identifier	Reject missing evidence

The guard relies on the curated evidence table; it cannot determine that an arbitrary external assertion is true. A separately verified availability example in an authored fixture tests the interface only. No such first-availability evidence is established in this cohort.

Validation. Source/method hashes for 25 dependencies were committed before the first comparison command. Native flat/API reconciliation passed 247,156 finite conditions; retained PDF/hash/page checking passed 712 automated conditions plus the manual native-page cells. Saved-projection checks passed 720 conditions; independent SQLite joins, CASE logic, calendar arithmetic and exercise expectations passed 152. Twenty-six authored fixture tests cover invalid/ambiguous dates, leap days, conflicting roles, empty denominators and unsupported conversions. Finite passing checks do not establish universal correctness.

Reproduce and use

Two separate full-history checkouts, CPython 3.10.11 and 3.12.14, reproduce all four analysis/exercise result files byte for byte, with 13 source projection files unchanged in each. This saved-data replay does not redownload native sources or independently repeat semantic adjudication.

```
python -m pip install -r requirements.txt
python src/replay.py --out ../recall-audit-replay
```

The repository includes all twelve cases, every planned report range, the sixty-row source adjudication, the local guard, authored fixtures and the original figures. Figure checks bind 36 actual artists to results and verify source coordinates, numeric labels and nine asset hashes through 190 finite conditions. The published manuscript and figure hashes are scoped explicitly.

Operational use. A data engineer can use this package to review a proposed recall label table before training or reporting turnaround times. Keep the notice type and planned/reported state as separate fields. Preserve precise source identifiers and channel distinctions. Investigate a source discrepancy rather than filling it with a convenient date. This implementation is local; no production users, measured workflow savings or deployed integration are claimed.

Limits and evidence still needed

These twelve early numbered 2023 campaigns are a finite convenience cohort, with related documents and repeated model records. Counts cannot estimate national reliability, safety performance or a remedy-time distribution. Conservative rules can leave a real event unresolved. Source equality does not verify mailing delivery, notification receipt or first repair availability.

The API and flat file are captured current sources. Amended documents do not reconstruct immutable original filings, first-byte availability or what a forecaster could have known at the time. Manufacturer statements, sample letters and planned fields remain attributed assertions. Physical first availability is unknown for all 12 cases; it is never filled from ODATE.

A predictive follow-up needs a defensible event definition, independently documented labels and feature snapshots at the prediction origin. A prospective guard evaluation needs a monitored workflow and recorded error handling. Neither is supplied by this descriptive audit. There is no causal, clinical, safety, financial-benefit or new-estimator claim.

Primary sources and publication boundary

NHTSA, Datasets and APIs; recall flat-file dictionary; campaign search and associated documents; Terms of Use. Exact campaign API URLs, native PDF URLs and source hashes are in the data manifests. The fixed capture and necessary factual projection are attributed; code uses MIT.

Full native archives, PDFs, contact blocks and vehicle identifiers remain private. This is an independent program manuscript, not a formal-publication submission. Azure optimization and other formal-publication projects are excluded from this repository.